How to use CloudSim in Eclipse

CloudSim is written in Java. The knowledge you need to use CloudSim is basic Java programming and some basics about cloud computing. Knowledge of programming IDEs such as Eclipse or NetBeans is also helpful. It is a library and, hence, CloudSim does not have to be installed. Normally, you can unpack the downloaded package in any directory, add it to the Java classpath and it is ready to be used. Please verify whether Java is available on your system.

To use CloudSim in Eclipse:

1. Download CloudSim installable files from *https://code.google.com/p/cloudsim/downloads/list and unzip*
2. Open Eclipse
3. Create a new Java Project: *File -> New*
4. Import an unpacked CloudSim project into the new Java Project
5. The first step is to initialise the CloudSim package by initialising the CloudSim library, as follows:

```
CloudSim.init(num_user, calendar, trace_flag)
```

6. Data centres are the resource providers in CloudSim; hence, creation of data centres is a second step. To create *Datacenter*, you need the *DatacenterCharacteristics* object that stores the properties of a data centre such as architecture, OS, list of machines, allocation policy that covers the time or spaceshared, the time zone and its price:

```
Datacenter datacenter9883 = new Datacenter(name,
characteristics, new VmAllocationPolicySimple(hostList),
storageList, 0);
```

7. The third step is to create a broker:

```
DatacenterBroker broker = createBroker();
```

8. The fourth step is to create one virtual machine unique ID of the VM, userId ID of the VM's owner, mips, number Of Pes amount of CPUs, amount of RAM, amount of bandwidth, amount of storage, virtual machine monitor, and *cloudletScheduler* policy for cloudlets:

```
Vm vm = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size,
vmm, new CloudletSchedulerTimeShared())
```

9. Submit the VM list to the broker:

```
broker.submitVmList(vmlist)
```

10. Create a cloudlet with length, file size, output size, and utilisation model:

```
Cloudlet cloudlet = new Cloudlet(id, length, pesNumber,
fileSize, outputSize, utilizationModel, utilizationModel,
utilizationModel)
```

11. Submit the cloudlet list to the broker:

```
broker.submitCloudletList(cloudletList)
```

12. Start the simulation:

```
CloudSim.startSimulation()
```

```
Sample Output from the Existing Example:
Starting CloudSimExample1...
Initialising...
Starting CloudSim version 3.0
Datacenter_0 is starting...
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>null
Broker is starting...
Entities started.
0.0: Broker: Cloud Resource List received with 1 resource(s)
0.0: Broker: Trying to Create VM #0 in Datacenter_0
0.1: Broker: VM #0 has been created in Datacenter #2, Host #0
0.1: Broker: Sending cloudlet 0 to VM #0
400.1: Broker: Cloudlet 0 received
400.1: Broker: All Cloudlets executed. Finishing...
400.1: Broker: Destroying VM #0
Broker is shutting down...
Simulation: No more future events
CloudInformationService: Notify all CloudSim entities for
shutting down.
Datacenter_0 is shutting down...
Broker is shutting down...
Simulation completed.
Simulation completed.
========== OUTPUT ==========
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start
Time   Finish Time
      0             SUCCESS     2             0       400
0.1             400.1
*****Datacenter: Datacenter_0*****
User id      Debt
     3            35.6
**********************************
CloudSimExample1 finished!
```

References

[1] *https://code.google.com/p/cloudsim/wiki/FAQ*

[2] ‘CloudSim: A toolkit for modelling and simulation of cloud computing environments and evaluation of resource provisioning algorithms’ by Rodrigo N Calheiros, Rajiv Ranjan, Anton Beloglazov, C´esar A F De Rose and Rajkumar Buyya

By: Mitesh Soni

The author is a technical lead at iGate Global Solutions Limited. He is in the Cloud Services (Research and Innovation) Group and loves to write about new technologies. Blogs: *http://clean-clouds.com* and *https://abcdofcloud.wordpress.com*